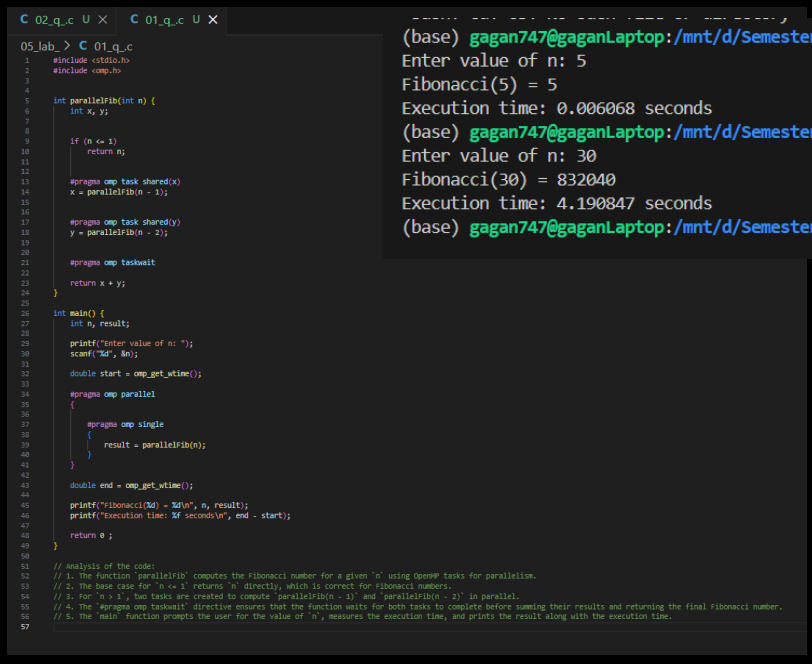
HPC – Lab _ 05 : (U23AI060)

Problem_01 :



```
#include <stdio.h>
#include <omp.h>

int parallelFib(int n) {
    int x, y;


    if (n <= 1)
        return n;


    #pragma omp task shared(x)
    x = parallelFib(n - 1);


    #pragma omp task shared(y)
    y = parallelFib(n - 2);


    #pragma omp taskwait

    return x + y;
}

int main() {
    int n, result;

    printf("Enter value of n: ");
    scanf("%d", &n);

    double start = omp_get_wtime();

    #pragma omp parallel
    {

        #pragma omp single
        {
            result = parallelFib(n);
        }
    }

    double end = omp_get_wtime();

    printf("Fibonacci(%d) = %d\n", n, result);
    printf("Execution time: %f seconds\n", end - start);

    return 0 ;
}

// Analysis of the code:
// 1. The function `parallelFib` computes the Fibonacci number for a given `n` using OpenMP tasks for parallelism.
// 2. The base case for `n <= 1` returns `n` directly, which is correct for Fibonacci numbers.
// 3. For `n > 1`, two tasks are created to compute `parallelFib(n - 1)` and `parallelFib(n - 2)` in parallel.
// 4. The `#pragma omp taskwait` directive ensures that the function waits for both tasks to complete before summing their results and returning the final Fibonacci number.
// 5. The `main` function prompts the user for the value of `n`, measures the execution time, and prints the result along with the execution time.
```

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/05_lab_$ ./p1
Enter value of n: 5
Fibonacci(5) = 5
Execution time: 0.006068 seconds
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/05_lab_$ ./p1
Enter value of n: 30
Fibonacci(30) = 832040
Execution time: 4.190847 seconds
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/05_lab_$
```

Problem_02 :

```c
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

//  BST ke nodes ka recursive sum parallel nested tasks se nikalna tha
//  aur variable scoping aur synchronization (taskwait vs barrier)
//  ka behaviour observe karna tha.

typedef struct Node {
    int data;
    struct Node *left, *right;
} Node;

Node* createNode(int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = value;
    newNode->left = newNode->right = NULL;
    return newNode;
}

Node* insert(Node* root, int value) {
    if (root == NULL) return createNode(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else
        root->right = insert(root->right, value);

    return root;
}

int parallelBSTSum(Node* root) {
    if (root == NULL)
        return 0;

    int leftSum = 0, rightSum = 0;

    #pragma omp task shared(leftSum) if(root->left != NULL)
    {
        leftSum = parallelBSTSum(root->left);
    }

    #pragma omp task shared(rightSum) if(root->right != NULL)
    {
        rightSum = parallelBSTSum(root->right);
    }

    #pragma omp taskwait
```

```
    return root->data + leftSum + rightSum;
}

int main() {
    Node* root = NULL;

    int values[10] = {5,3,7,2,4,6,8,1,9,10};

    for(int i=0;i<10;i++)
        root = insert(root, values[i]);

    int total = 0;

    #pragma omp parallel
    {
        #pragma omp single
        {
            total = parallelBSTSum(root);
        }
    }

    printf("BST Sum = %d\n", total);

    return 0;
}
```

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/05_lab_$ gcc -fopenmp 02_q_.c -o p2
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/05_lab_$ ./p2
BST Sum = 55
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/05_lab_$
```

```
// Analysis of the code:
// 1. The code defines a binary search tree (BST) with a structure `Node` and functions to create nodes and insert values.
// 2. The `parallelBSTSum` function computes the sum of all nodes in the
BST using OpenMP tasks for parallelism.
// 3. For each node, two tasks are created to compute the sum of the left and right subtrees in parallel, with a condition to avoid creating tasks for NULL children.
// 4. The `#pragma omp taskwait` directive ensures that the function waits for both tasks to complete before summing their results and returning the total sum for the current node.
// 5. The `main` function constructs a BST with predefined values, computes the total sum of the nodes in parallel, and prints the result.
```

Problem _ 03 :

C 02_q_.c U | C 03_q_.c U | C 01_q_.c U

05_lab_ > C 03_q_.c

```c
#include <stdio.h>
#include <omp.h>

#define SIZE 10
int arr[SIZE] = {1,2,3,4,5,6,7,8,9,10};

int parallelSum(int low, int high) {

    if (low == high)
        return arr[low];

    int mid = (low + high) / 2;
    int left=0, right=0;

    #pragma omp task shared(left)
    left = parallelSum(low, mid);

    #pragma omp task shared(right)
    right = parallelSum(mid+1, high);

    #pragma omp taskwait

    printf("Thread %d merged range [%d-%d]\n",
           omp_get_thread_num(), low, high);

    return left + right;
}

int main() {
    int result;

    #pragma omp parallel
    {
        #pragma omp single
        result = parallelSum(0, SIZE-1);
    }

    printf("Sum = %d\n", result);
}

// Analysis of the code:
// 1. The function `parallelSum` computes the sum of elements in the array `
// `arr` between the indices `low` and `high` using a divide-and-conquer approach with OpenMP tasks for parallelism.
// 2. If `low` is equal to `high`, it means we are at
// a single element, and the function returns that element directly.
// 3. For larger ranges, the function calculates the midpoint and creates two tasks to compute the sum of the left and right halves of the range in parallel.
// 4. The `#pragma omp taskwait` directive ensures that the function waits for both tasks to complete before summing their results and returning the total sum for the current range.
// 5. The `main` function initializes the parallel region and calls `parallelSum` for the entire array, then prints the final result.
```

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/05_lab_$ gcc -fopenmp 03_q_.c -o p3
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/05_lab_$ ./p3
Thread 4 merged range [8-9]
Thread 10 merged range [0-1]
Thread 5 merged range [0-2]
Thread 9 merged range [3-4]
Thread 13 merged range [0-4]
Thread 14 merged range [5-6]
Thread 6 merged range [5-7]
Thread 12 merged range [5-9]
Thread 11 merged range [0-9]
Sum = 55
```

Other part of problem _ 03 :

05_lab_ > C 03_b_q_.c

```c
#include <stdio.h>
#include <omp.h>

#define SIZE 10
int arr[SIZE];

int main() {

    int sum = 0;

    #pragma omp parallel
    {

        #pragma omp for nowait
        for(int i=0;i<SIZE;i++)
            arr[i] = i+1;


        #pragma omp taskwait


        #pragma omp for reduction(+:sum)
        for(int i=0;i<SIZE;i++)
            sum += arr[i];
    }

    printf("Sum = %d\n", sum);
}

// Analysis of the code:
// 1. The code initializes an array `arr` of size `SIZE` with values
// from 1 to `SIZE` in parallel using an OpenMP `for` loop with the `nowait` clause to allow the next section to start without waiting for the initialization to complete.
// 2. The `#pragma omp taskwait` directive ensures that all tasks created in
   // the previous section are completed before proceeding to the next section, which computes the sum of the elements in the array.
// 3. The second `for` loop uses the `reduction` clause to safely
   // compute the sum of the elements in parallel, ensuring that the updates to `sum` are done atomically to avoid race conditions
// 4. Finally, the computed sum is printed to the console.
// 5. The `main` function demonstrates the use of OpenMP for parallelizing both the initialization of the array
//  and the computation of the sum, showcasing how to manage dependencies between tasks using `taskwait`.
```

```
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/05_lab_$ gcc -fopenmp 03_b_q_.c  -o p3b
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/05_lab_$ ./p3b
Sum = 55
(base) gagan747@gaganLaptop:/mnt/d/Semester_06_/HPC/LAB/05_lab_$
```